

Git - Guia completo de comandos básicos

Material didático para estudar Git no dia a dia: comandos, fluxos, analogias, boas práticas e resolução de erros comuns.

Objetivo: entender o que cada comando faz, quando usar e como evitar problemas comuns ao trabalhar sozinho ou em equipe.

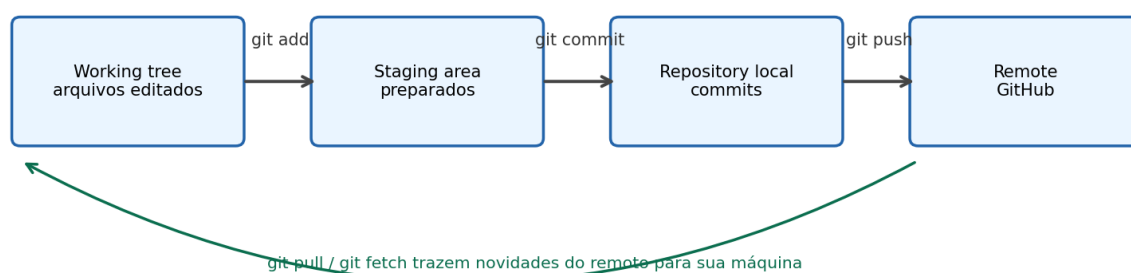


Figura 1 - Ciclo básico do Git: editar, preparar, salvar no histórico e enviar para o remoto.

1. Visão geral: o que é Git?

Git é um sistema de controle de versão. Ele registra o histórico do seu projeto por meio de commits. Cada commit funciona como uma foto do estado do código em determinado momento. Com isso, você consegue voltar versões, comparar alterações, criar branches e trabalhar em equipe com mais segurança.

Analogia: pense no Git como um videogame com vários pontos de salvamento. Cada commit é um save. Se uma fase der errado, você consegue comparar, corrigir ou voltar para um save anterior.

A diferença entre Git e GitHub é simples: Git é a ferramenta instalada na sua máquina; GitHub é uma plataforma online que hospeda repositórios remotos. Você pode usar Git sem GitHub, mas o GitHub facilita colaboração, backup e revisão de código.

2. Termos essenciais

Termo	Significado	Exemplo
Repositório	Pasta do projeto controlada pelo Git.	git init
Commit	Registro de uma alteração salva no histórico.	git commit -m "mensagem"
Branch	Linha de desenvolvimento independente.	main, login, feature/produto
Remote	Apelido para um repositório remoto, geralmente no GitHub.	origin
Working tree	Arquivos do projeto na sua máquina.	onde você edita o código
Staging area	Área de preparação antes do commit.	git add .
Merge	Juntar alterações de uma branch em outra.	git merge login
Pull	Buscar e aplicar alterações do remoto.	git pull origin main
Push	Enviar commits locais para o remoto.	git push origin main

3. Configuração inicial

Antes de criar commits, configure seu nome e e-mail. Essas informações aparecem no histórico do projeto.

```
git config --global user.name "Pedro Carrara"
git config --global user.email "seu-email@email.com"
```

Para conferir as configurações atuais:

```
git config --list
```

Use --global para aplicar a configuração para todos os projetos da sua máquina. Sem --global, a configuração vale apenas para o repositório atual.

4. Criando ou baixando repositórios

Comando	Quando usar	Exemplo
git init	Transformar uma pasta comum em um repositório Git.	git init
git clone URL	Baixar um repositório remoto para sua máquina.	git clone git@github.com:usuario/projeto.git
git remote -v	Ver quais remotos estão configurados.	git remote -v
git remote add origin URL	Adicionar o remoto chamado origin.	git remote add origin git@github.com:usuario/projeto.git
git remote remove origin	Remover o remoto origin.	git remote remove origin
git remote set-url origin URL	Trocar a URL do remoto origin.	git remote set-url origin git@github.com:usuario/novo.git

origin é apenas um apelido. Por padrão, usamos origin para representar o repositório principal no GitHub. Em vez de digitar a URL inteira toda vez, você usa o apelido origin.

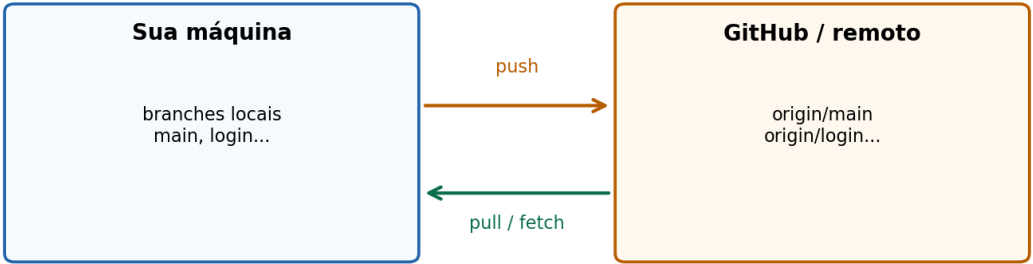


Figura 2 - Relação entre sua máquina e o repositório remoto.

5. O ciclo básico: status, add, commit e log

Esse é o fluxo mais usado no dia a dia. Você altera arquivos, confere o status, prepara as alterações, cria um commit e depois envia para o GitHub.

Comando	Quando usar	Exemplo
git status	Ver o estado atual do projeto. Mostra arquivos modificados, novos e preparados.	git status
git add arquivo	Preparar um arquivo específico para o próximo commit.	git add Produto.java
git add .	Preparar todas as alterações da pasta atual.	git add .
git commit -m "msg"	Salvar as alterações preparadas no histórico local.	git commit -m "Adiciona login"
git log	Ver o histórico de commits.	git log
git log --oneline	Ver o histórico de forma resumida.	git log --oneline
git diff	Ver alterações ainda não preparadas.	git diff
git diff --staged	Ver alterações que já estão no staging.	git diff --staged

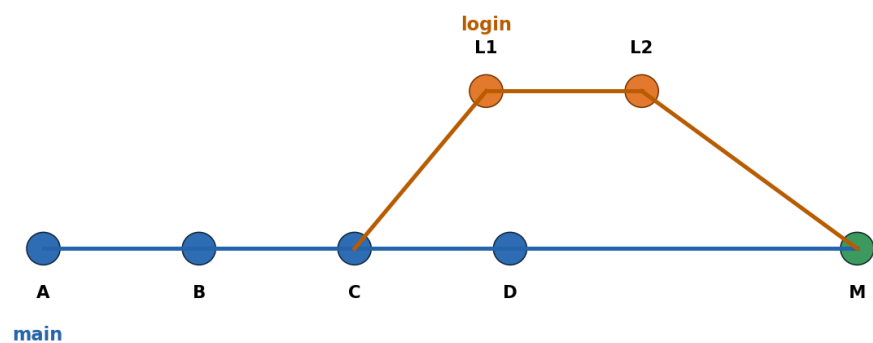
Fluxo básico recomendado:

```
git status
git add .
git commit -m "Descreva a alteração"
git push
```

Analogia: o git add é como colocar itens em uma caixa. O git commit é lacrar a caixa e etiquetar com uma descrição. O git push é enviar essa caixa para o GitHub.

6. Branches: criando, trocando e apagando

Branch é uma linha de desenvolvimento. Em projetos reais, a branch main costuma representar a versão principal, enquanto branches como login, feature/pagamento ou fix/erro-api guardam alterações específicas.



Fluxo para trazer login para main:

```
git switch main | git pull origin main | git merge login | git push origin main
```

Figura 3 - A branch login nasce da main e depois volta para main por merge.

Comando	Quando usar	Exemplo
git branch	Listar branches locais.	git branch
git switch nome	Entrar em uma branch existente.	git switch main

Comando	Quando usar	Exemplo
git switch -c nome	Criar e entrar em uma branch nova.	git switch -c login
git checkout -b nome	Forma antiga, ainda muito usada, para criar e entrar em uma branch.	git checkout -b login
git branch -d nome	Apagar branch local já integrada/mergeada.	git branch -d login
git branch -D nome	Forçar a exclusão de uma branch local. Use com cuidado.	git branch -D login
git branch -M main	Renomear a branch atual para main.	git branch -M main

A documentação oficial apresenta switch e checkout como formas de trocar de branch. Atualmente, git switch costuma ser mais claro para iniciantes, porque seu foco é apenas trocar/criar branches.

7. Fluxo recomendado para nova funcionalidade

Quando for criar uma funcionalidade nova, evite trabalhar direto na main. Crie uma branch separada. Isso reduz risco e deixa o histórico mais organizado.

```
git switch main
git pull origin main
git switch -c login
# faça as alterações
git add .
git commit -m "Adiciona funcionalidade de login"
git push -u origin login
```

Depois, normalmente você abre um Pull Request no GitHub para revisar e juntar a branch login na main. Em estudos ou projetos pessoais, você também pode fazer o merge localmente.

8. Merge: trazendo alterações de uma branch para outra

O git merge junta o histórico de uma branch dentro da branch em que você está. O detalhe mais importante é: o merge sempre acontece na branch atual.

Exemplo: você está na main e quer trazer a branch login para dentro dela:

```
git switch main
git pull origin main
git merge login
git push origin main
```

Esse fluxo significa: entre na main, atualize a main com o remoto, junte as alterações da login e envie a main atualizada para o GitHub.

Cuidado: se você estiver na branch errada, o merge vai para o lugar errado. Antes de fazer merge, sempre rode git branch ou git status para conferir a branch atual.

9. Repositório remoto: fetch, pull e push

fetch busca; pull busca e aplica

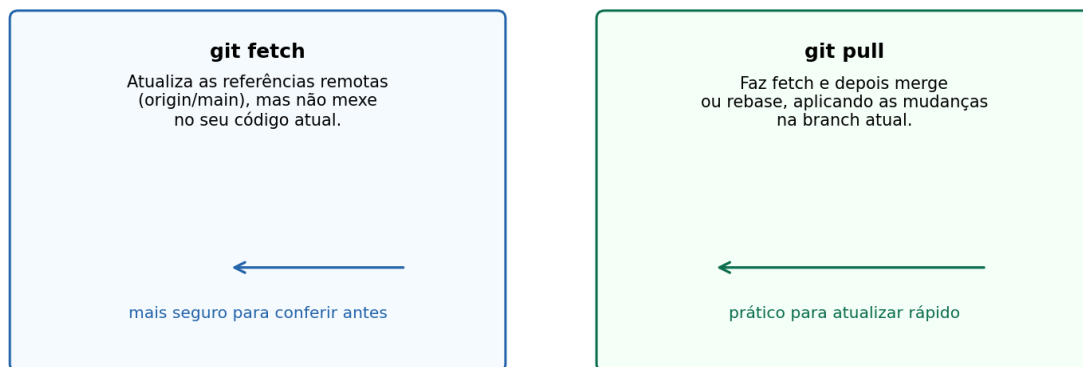


Figura 4 - Diferença entre fetch e pull.

Comando	Quando usar	Exemplo
git fetch origin	Buscar informações novas do remoto sem aplicar no seu código.	git fetch origin
git pull origin main	Buscar e aplicar alterações da main remota na branch atual.	git pull origin main
git push origin main	Enviar commits da sua main local para a main remota.	git push origin main
git push -u origin main	Enviar a branch e configurar o upstream. Depois disso, git push e git pull funcionam sem informar origem e branch.	git push -u origin main
git push -u origin login	Enviar uma branch nova para o GitHub e ligar a branch local à remota.	git push -u origin login

Importante: git pull origin main pega a main do remoto e aplica na branch em que você está agora. Se você estiver em login, ele traz main para dentro de login. Se estiver em main, ele atualiza a main local.

10. Pull explicado de forma simples

O git pull é um atalho para buscar alterações do remoto e aplicar na sua branch atual. Na prática, ele faz algo parecido com:

```
git fetch origin
```

```
git merge origin/main
```

Use git pull quando outra pessoa alterou o repositório remoto ou quando você trabalhou em outro computador e precisa trazer as novidades para a máquina atual.

Exemplo clássico para atualizar sua main:

```
git switch main  
git pull origin main
```

Exemplo para atualizar sua branch de trabalho com o que há de novo na main:

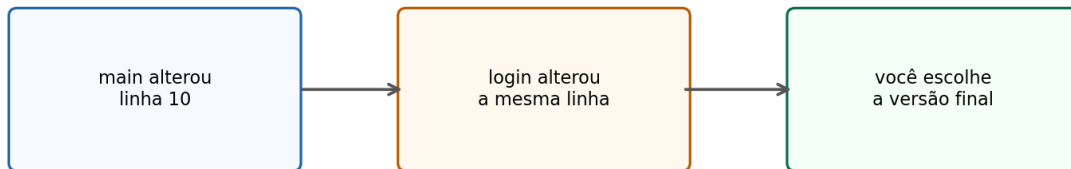
```
git switch login  
git pull origin main
```

No segundo caso, você está dizendo: “estou trabalhando em login, mas quero trazer para ela as atualizações recentes da main remota”.

11. Resolvendo conflitos

Conflito acontece quando o Git não consegue decidir sozinho qual alteração manter. Isso geralmente ocorre quando duas branches modificam o mesmo trecho do mesmo arquivo.

Conflito: duas alterações no mesmo trecho



Depois de resolver: `git add arquivo -> git commit`

Figura 5 - Em um conflito, você escolhe manualmente a versão final do código.

Ao abrir o arquivo com conflito, você pode ver algo parecido com:

```

<<<<<< HEAD
versão da sua branch atual
=====
versão da outra branch
>>>>>> login
  
```

Você deve editar o arquivo, remover os marcadores e deixar apenas o código correto. Depois, finalize assim:

```

git add arquivo-com-conflito
git commit
  
```

Se o conflito aconteceu durante um merge, o commit finaliza o merge. Se aconteceu durante um pull, o raciocínio é parecido, porque pull pode executar um merge por baixo dos panos.

12. Desfazendo alterações com segurança

Comando	Quando usar	Exemplo
<code>git restore arquivo</code>	Desfazer alteração ainda não commitada em um arquivo.	<code>git restore Produto.java</code>
<code>git restore --staged arquivo</code>	Tirar um arquivo do staging sem apagar a alteração do arquivo.	<code>git restore --staged Produto.java</code>
<code>git reset --soft HEAD~1</code>	Desfazer o último commit, mantendo as alterações preparadas.	<code>git reset --soft HEAD~1</code>
<code>git reset --mixed HEAD~1</code>	Desfazer o último commit e manter as alterações no working tree.	<code>git reset --mixed HEAD~1</code>
<code>git reset --hard HEAD~1</code>	Desfazer o último commit e apagar as alterações. Perigoso.	<code>git reset --hard HEAD~1</code>
<code>git revert HASH</code>	Criar um novo commit que desfaz um commit anterior. Mais seguro para commits já enviados.	<code>git revert a1b2c3d</code>

Regra prática: se o commit já foi enviado para o GitHub e outras pessoas podem ter baixado, prefira `git revert`. Evite reescrever histórico compartilhado com `reset hard`.

13. Stash: guardando alterações temporariamente

O git stash guarda alterações não commitadas em uma “gaveta temporária”. É útil quando você está no meio de uma tarefa, mas precisa trocar de branch sem criar commit ainda.

```
git stash
git switch main
# resolva algo urgente
git switch login
git stash pop
```

Comando	Quando usar	Exemplo
git stash	Guardar alterações temporariamente.	git stash
git stash list	Listar alterações guardadas.	git stash list
git stash pop	Aplicar o stash mais recente e removê-lo da lista.	git stash pop
git stash apply	Aplicar o stash, mas manter ele guardado.	git stash apply
git stash drop	Apagar um stash.	git stash drop stash@{0}

14. Rebase: quando aparece e para que serve

O git rebase reaplica commits de uma branch sobre outra base. Ele é usado para deixar o histórico mais linear. Em times, é comum usar rebase para atualizar sua branch de funcionalidade com a main antes de abrir ou atualizar um Pull Request.

```
git switch login
git fetch origin
git rebase origin/main
```

Analogia: imagine que você começou seu trabalho em cima de uma versão antiga da main. O rebase pega seus commits e recoloca eles em cima da versão mais nova da main.

Cuidado: evite fazer rebase em branches compartilhadas se outras pessoas já estão usando a mesma branch. Rebase reescreve o histórico.

15. .gitignore: ignorando arquivos que não devem ir para o Git

O arquivo .gitignore define arquivos e pastas que o Git deve ignorar. Ele é usado para evitar subir arquivos temporários, binários, senhas, caches e configurações locais.

```
# exemplo de .gitignore para Java/Spring
target/
*.class
.env
.idea/
.vscode/
*.log
```

Em projetos Java com Maven, por exemplo, a pasta target/ é gerada automaticamente durante o build. Normalmente ela não deve ser versionada.

16. Tags: marcando versões importantes

Tags são marcadores usados para identificar versões importantes, como releases.

```
git tag v1.0.0
git push origin v1.0.0
```

Para listar tags:

```
git tag
```

17. Comandos úteis de inspeção

Comando	Quando usar	Exemplo
git status	Primeiro comando para entender o estado atual.	git status
git branch -a	Listar branches locais e remotas.	git branch -a
git log --oneline --graph --all	Ver histórico em formato gráfico no terminal.	git log --oneline --graph --all
git show HASH	Ver detalhes de um commit específico.	git show a1b2c3d
git blame arquivo	Ver qual commit/autor alterou cada linha.	git blame Produto.java

18. Erros comuns e como resolver

Erro	O que significa	Como resolver
fatal: not a git repository	Você não está dentro de uma pasta versionada pelo Git.	Entre na pasta correta ou use git init.
remote origin already exists	O remoto origin já foi cadastrado.	Use git remote -v, git remote remove origin ou git remote set-url origin URL.
src refspec main does not match any	A branch main não existe localmente ou não há commits ainda.	Confira git branch. Talvez seja master ou falte fazer commit.
current branch has no upstream	Sua branch local ainda não está ligada a uma branch remota.	Use git push -u origin nome-da-branch.
merge conflict	Git encontrou alterações incompatíveis no mesmo trecho.	Edite os arquivos, remova marcadores, git add e git commit.

19. Fluxos prontos para copiar

Criar um projeto novo e enviar para o GitHub:

```
git init
git add .
git commit -m "Commit inicial"
git branch -M main
git remote add origin git@github.com:usuario/projeto.git
git push -u origin main
```

Começar uma funcionalidade nova:

```
git switch main
git pull origin main
git switch -c feature/nome-da-funcionalidade
```

Salvar e enviar uma branch nova:

```
git status
git add .
git commit -m "Implementa funcionalidade"
git push -u origin feature/nome-da-funcionalidade
```

Trazer uma branch login para dentro da main:

```
git switch main
git pull origin main
git merge login
git push origin main
```

20. Resumo mental para não se perder

Objetivo	Comando
Quero saber onde estou	git status e git branch
Quero preparar alterações	git add .
Quero salvar no histórico local	git commit -m "mensagem"
Quero enviar para o GitHub	git push
Quero baixar atualizações	git pull
Quero criar uma branch	git switch -c nome
Quero entrar em outra branch	git switch nome
Quero juntar uma branch na atual	git merge nome
Quero desfazer alteração não commitada	git restore arquivo
Quero guardar alteração temporária	git stash

Dica final: antes de comandos que alteram histórico ou juntam branches, rode git status. Esse hábito evita grande parte dos erros de iniciantes.

Referências de estudo

Conteúdo revisado com base em conceitos estáveis de Git e documentação de referência como Git SCM Book, Git Cheat Sheet, GitHub Docs e guias de fluxo com branches. As explicações foram adaptadas para estudo prático e linguagem didática.